

Komplett Lathund: Test kap. 1–4 (Teori, Syntax & Loopar)

1. God Praxis & Kodstil (PEP 8)

- **Namngivning:** Använd engelska, förklarande namn i `snake_case` (t.ex. `mean_velocity` istället för `v` eller `mean_vel`).
- **Hårdkodning:** Undvik värden/litteraler spridda i koden. Definiera dem en gång i tydligt namngivna variabler längst upp.
- **Globala variabler:** Undvik nyckelordet `global`. Skicka in data via funktionens parametrar och returnera värden med `return`.
- **Dokumentsträng (Docstring):** Skriv Beskrivning av funktionen" direkt under funktionshuvudet.

2. f-strings (Formaterade strängar): Vad betyder f och hur är syntaxen?

Bokstaven `f` framför en sträng står för **formatted string** (f-string).

Det gör att du kan bädda in Python-variabler och uttryck direkt inuti texten genom att omsluta dem med mäsvingar `{...}`. Python beräknar automatiskt värdet inuti `{...}` och omvandlar det till text vid utskrift.

```
namn = "Karolina"
alder = 22

# SÄTT 1: Med f-string (REN, TYDLIG OCH REKOMMENDERAD SYNTAX)
print(f"Hej! Jag heter {namn} och ar {alder} ar gammal.")

# SÄTT 2: Gamla / omständliga sättet (utan f-string)
print("Hej! Jag heter " + namn + " och ar " + str(alder) + " ar gammal.")

--- VAD DU KAN GÖRA INUTI { ... } ---

# 1. Direkt beräkning/matematik:
print(f"Om 5 ar ar jag {alder + 5} ar.") # Skriver ut: Om 5 ar ar jag 27 ar.

# 2. Anropa metoder eller funktioner:
print(f"Namn i stora bokstaver: {namn.upper()}") # Skriver ut: KAROLINA

# 3. Formatera flyttal/decimaler (t.ex. avrunda till 2 decimaler med {:.2f}):
pi = 3.14159265
print(f"Pi med tva decimaler: {pi:.2f}") # Skriver ut: 3.14

# 4. Formatera dictionaries:
bruker = {"namn": "Anna", "alder": 30}
# OBS: Anvand enkelfnuttar (') inuti mäsvingarna om strängen har dubbelfnuttar (") på utsidan!
print(f"Anvandaren heter {bruker['namn']}.")
```

3. Funktioner vs. Metoder med Punkt (.funktion())

- **Vanlig funktion** – `funktion(objekt)`: Du skickar in objektet som ett argument i funktionen.

```
len(min_lista) # Beräknar längden på listan
print("Hej")   # Skriver ut texten
type(x)        # Returnerar datatypen för x
```

- **Metod med punkt** – `objekt.metod()`: Metoden "hör" till objektets datatyp. Du ber objektet utföra en handling på sig själv.

```
text = "katt"
text.upper() # Returnerar "KATT" (strängmetod)
lista.append("hund") # Lägger till i slutet av listan (listmetod)
lager.get("applen") # Hämtar värde säkert från dictionary
```

4. Syntax för Loopar (for, while & range)

- **Hur range() fungerar:** range(start, stop, step) genererar en sekvens av tal. **Viktigt:** Slutalet (stop) ingår **ALDRIG!**

- range(5) → 0, 1, 2, 3, 4
- range(2, 6) → 2, 3, 4, 5
- range(0, 10, 2) → 0, 2, 4, 6, 8 (steglängd 2)

- **for-loop:** Används när du vet hur många iterationer som ska köras, eller när du vill gå igenom varje element i en sekvens.

```
# 1. Gå igenom element i en lista
djur_lista = ["katt", "hund", "kanin"]
for djur in djur_lista:
    print("Djur:", djur)

# 2. Loopa med range(start, stop, step)
for i in range(0, 10, 2): # Går från 0 till 8 med steg 2
    print("Tal:", i)
```

- **while-loop med True, input() och break:** Används för kontinuerlig inmatning tills användaren aktivt avbryter (t.ex. genom att skriva "exit").

```
while True:
    svar = input("Ange ett ord (eller 'exit' för att avsluta): ")
    if svar.lower() == "exit":
        print("Avslutar programmet...")
        break # Hoppa direkt ut ur loopen
    print("Du skrev:", svar)
```

- Extra om for vs while loop. Användning av Index: for-loop omgjord till while-loop Detta exempel visar hur man går igenom en lista med index via en for-loop respektive hur man skriver om samma logik med en while-loop där indexet täcks med i = 0 och i += 1.

```
frukter = ["äpple", "banan", "päron"]

# SÄTT 1: for-loop med range(len(...))
for i in range(len(frukter)):
    print(f"Index {i}: {frukter[i]}")

# SÄTT 2: Omgjord till while-loop
i = 0 # 1. Starta räknaren på index 0
while i < len(frukter): # 2. Loopa så länge i är mindre än listans längd
    print(f"Index {i}: {frukter[i]}")
    i += 1 # 3. GLÖM INTE att öka räknaren varje varv!
```

- **break, continue och pass:**

- **break:** Avbryter loopen helt och hoppar ut.
- **continue:** Hoppa över resten av nuvarande varv och går till nästa iteration.
- **pass:** Platshållare för framtida kod (gör ingenting).

```
for tal in range(5):
    if tal == 2:
        continue # Hoppa över utskriften för talet 2
    if tal == 4:
        break # Avbryter loopen helt när talet är 4
    print(tal)
```

5. Nyckelordet in

Nyckelordet `in` används i två huvudsakliga sammanhang:

1. **Medlemskapstest (booleskt resultat):** Kontrollerar om ett element finns i en lista, sträng eller dictionary (returnerar `True` eller `False`).
2. **Iteration:** Används i `for`-loopar för att gå igenom sekvenser.

```
frukter = ["äpple", "banan", "päron"]

# 1. Medlemskapstest (blir True/False)
finns_banan = "banan" in frukter # True
har_bokstav = "a" in "katt"      # True
finns_nyckel = "applen" in {"applen": 5} # True

# 2. Iteration i for-loop
for frukt in frukter:
    print(frukt)
```

6. Operatörer, Modulo, Booleska Uttryck & Fallgropar

- **Modulo-operatören (%):** Beräknar **resten** vid heltalsdivision. Används ofta för att kontrollera om ett tal är jämnt eller udda.

```
tal = 7
if tal % 2 == 0:
    print("Talet är jämnt")
else:
    print("Talet är udda")
```

- **Uttryck som blir True eller False direkt:** Undvik onödiga `if/else`-satser för att returnera booleska värden.

```
# DÅLIG STIL:
def is_longer_bad(word):
    if len(word) > 8:
        return True
    else:
        return False

# BRA STIL (returnerar uttryckets booleska resultat direkt):
def is_longer(word):
    return len(word) > 8
```

- **Reserverade ord (Keywords):** Får ej användas som variabler: `and`, `if`, `for`, `True`, `False`, `None`, `break`, `continue`, `pass`, `def`, `return`, `with`.
- **Identifierare:** Måste börja med en bokstav eller understreck (`_`), aldrig med en siffra.
- **Vanliga operatorfel:** Potens skrivs med `**` (inte `^`), större än/lika med skrivs `>=` (inte `=>`), och jämförelse är `==` (inte `=`).
- **Flyttalsprecision:** Jämför inte flyttal med `==` (p.g.a. `0.1 + 0.2 != 0.3`). Använd en tolerans: `abs((0.1 + 0.2) - 0.3) < 1e-9`.

7. Teoribegrepp & Filhantering

- **Funktionsstruktur:**
 - *Huvud:* `def min_funktion(param1, param2=10):`
 - *Kropp:* Den indenterade koden i funktionen.
 - *Parametrar vs Argument:* Parametrar är variablerna i funktionens definition; Argument är de faktiska värdena som skickas vid anrop.
- **Paradigmer:** *Imperativ* (steg-för-steg ändring av tillstånd), *Funktionell* (rena funktioner utan sideeffekter), *OOP* (klasser och objekt med attribut/metoder).

- **Filhantering – with open(...) vs. enbart open(...):**

- Enbart `open()`: Kräver manuell `.close()`. Om programmet kraschar stängs inte filen.
- `with open(...)` (**REKOMMENDERAT**): Stänger filen automatiskt när blocket avslutas, vilket frigör resurser och säkrar sparandet.

```
# UTAN with (Dålig praxis - kräver fil.close()):
fil = open("brev.txt", "r")
innehall = fil.read()
fil.close()

# MED with (Bra praxis - stänger automatiskt):
with open("brev.txt", "r") as fil:
    innehall = fil.read()
```

- **System-strömmar:** `sys.stdin` (inmatning/tangentbord), `sys.stdout` (utmatning/konsol), `sys.stderr` (felmeddelanden).
- **Lokal kopia:** Använd `.copy()` för listor/objekt för att skapa en ny oberoende kopia i minnet så att ändringar inte påverkar originalet.

8. Snabbreferens för Datastrukturer

- **Listor (Muterbara/Ändringsbara):**

```
data = ["katt", 7, "joker"]
data.append("hund")      # Lägg till sist
data.insert(1, "apa")    # Lägg till på index 1
data.pop(2)              # Ta bort och returnera element på index 2
del data[0]              # Ta bort element på index 0
data.remove("joker")     # Ta bort första förekomsten av "joker"
data.reverse()           # Ändra listan på plats
baklänges = data[::-1]  # Ändra listan med slicing
```

- **Dictionaries (Uppslagstabeller):**

```
lager = {"applen": 10, "bananer": 5}
lager["bananer"] = 8      # Uppdatera värde
antal = lager.get("applen") # Hämtar värde säkert
for frukt, antal in lager.items(): # Loopa över nyckel och värde
    print(frukt, antal)
```